PROJECT 9

One-Prompt Strikes Back:

Sparse Mixture of Experts for Prompt-Based Continual Learning



Published as a conference paper at ICLR 2026

ONE-PROMPT STRIKES BACK: SPARSE MIXTURE OF EXPERTS FOR PROMPT-BASED CONTINUAL LEARNING

Minh Le[*1], **Bao Ngoc Dao**[*4], **Huy Nguyen**[2], **Quyen Tran**[5], **Anh Nguyen**[3], **Nhat Ho**[2]

[1] Trivita AI [2] The University of Texas at Austin [3] Northeastern University

[4] Hanoi University of Science and Technology [5] Rutgers University

ABSTRACT

Prompt-based methods have recently gained prominence in Continual Learning (CL) due to their strong performance and memory efficiency. A prevalent strategy in this paradigm assigns a dedicated subset of prompts to each task, which, while effective, incurs substantial computational overhead and causes memory requirements to scale linearly with the number of tasks. Conversely, approaches employing a single shared prompt across tasks offer greater efficiency but often suffer from degraded performance due to knowledge interference. To reconcile this trade-off, we propose **SMoPE**, a novel framework that integrates the benefits of both task-specific and shared prompt strategies. Inspired by recent findings on the relationship between Prefix Tuning and Mixture of Experts (MoE), SMoPE organizes a shared prompt into multiple "prompt experts" within a sparse MoE architecture. For each input, only a select subset of relevant experts is activated, effectively mitigating interference. To facilitate expert selection, we introduce a prompt-attention score aggregation mechanism that computes a unified proxy score for each expert, enabling dynamic and sparse activation. Additionally, we propose an adaptive noise mechanism to encourage balanced expert utilization while preserving knowledge from prior tasks. To further enhance expert specialization, we design a prototype-based loss function that leverages prefix keys as implicit memory representations. Extensive experiments across multiple CL benchmarks demonstrate that SMoPE consistently outperforms task-specific prompt methods and achieves performance competitive with state-of-the-art approaches, all while significantly reducing parameter counts and computational costs. Our code is publicly available at https://github.com/Minhchuyentoancbn/SMoPE.

GROUP 18

Poorvai Chandrasen
Palak Kumari
Shruti Wadatkar

WHAT IS

Sparse — SMALL subset out of multiple (ranked by topk)

MoE — A mix of outputs from GATES (routers) that score each input

for

Prompt — **small**, lightweight set of **learnable parameters** appended to a pre-trained backbone model

based

CL — training setup where a neural network **incrementally** learns from a sequence of tasks over time

Problem Statement

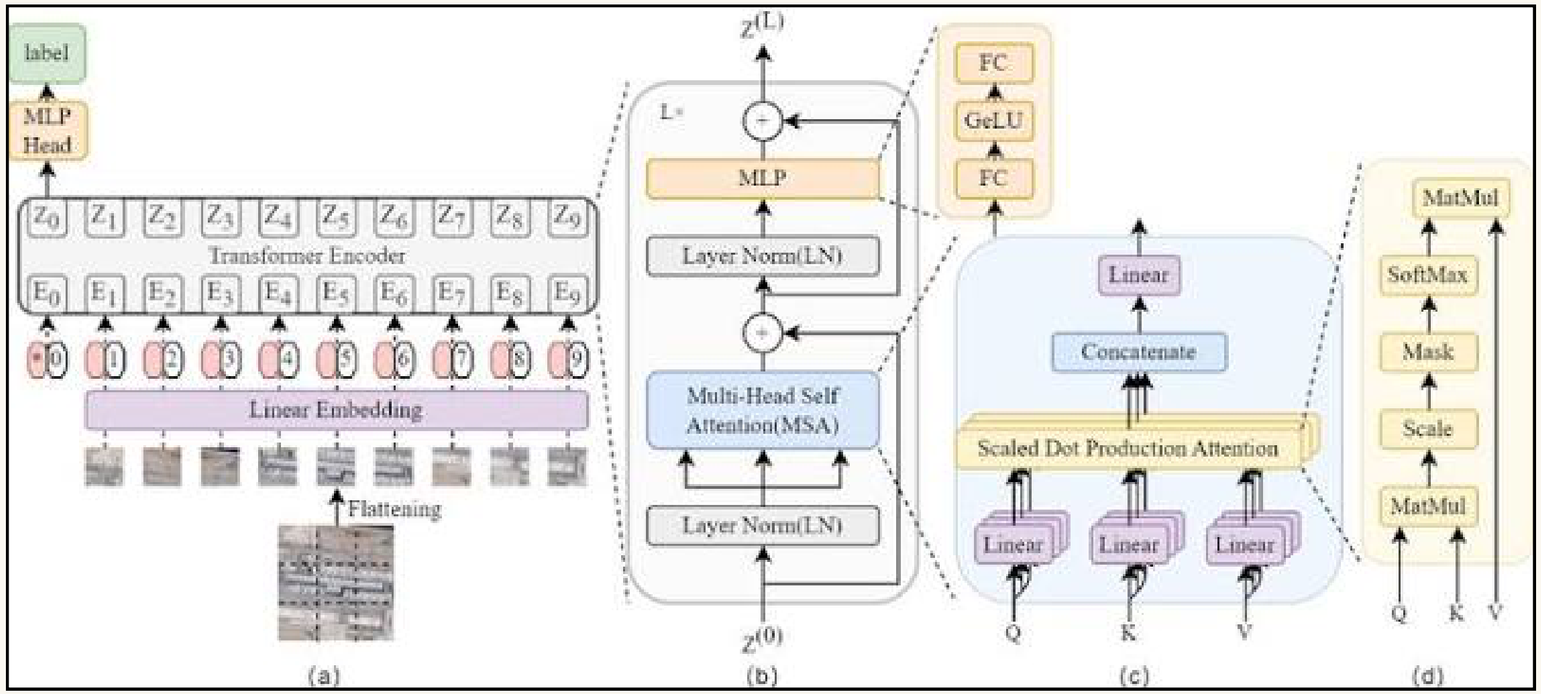
Dataset: CIFAR-100 for Image Classification

Tasks: 10 (each task has 10 classes from dataset)

Model: ViT (frozen Backbone) pretrained on ImageNet

Catch: Inputs from previous task are not available after training

Desired Outcome: High Accuracy on all tasks



Source: Vision Transformer With Contrastive Learning for Remote Sensing Image Scene Classification - Scientific Figure on ResearchGate.

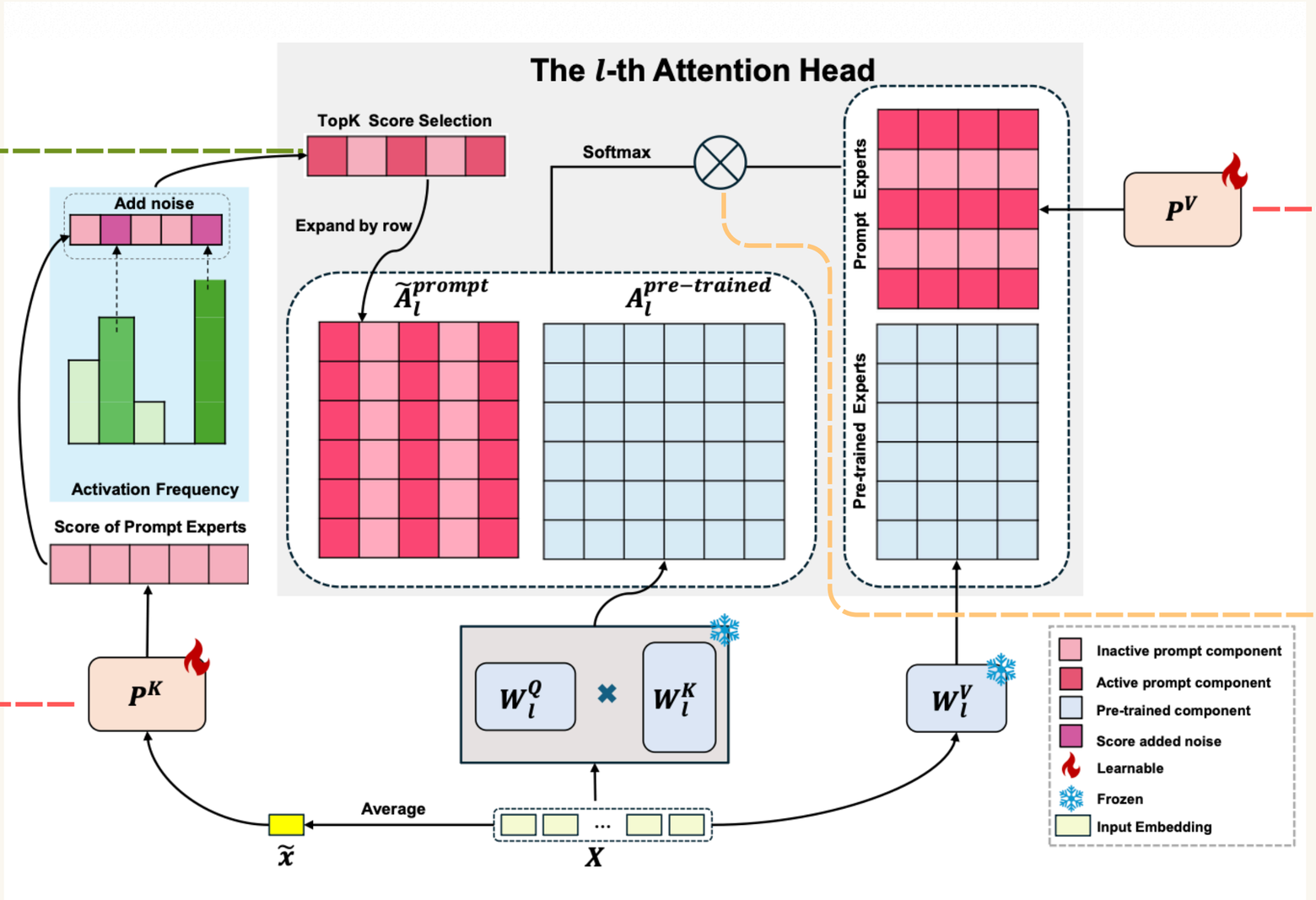
Solution: SMoPE MODEL Architecture

Sparse

Prompt

Prompt

MoE



Key mechanisms: SMoPE Model

Mixture of Experts Model

for $i = 1, \ldots, N$, $j = 1, \ldots, N$, and $j' = 1, \ldots, N_p$. With these definitions, the output of the i-th token in head l can be expressed as an MoE model:

$$\hat{\boldsymbol{h}}_{l,i} = \sum_{j=1}^{N} \frac{\exp(s_{i,j}(\boldsymbol{X}))}{\sum_{k=1}^{N} \exp(s_{i,k}(\boldsymbol{X})) + \sum_{k'=1}^{N_p} \exp(s_{i,N+k'}(\boldsymbol{X}))} f_j(\boldsymbol{X}) + \sum_{j'=1}^{N_p} \frac{\exp(s_{i,N+j'}(\boldsymbol{X}))}{\sum_{k=1}^{N} \exp(s_{i,k}(\boldsymbol{X})) + \sum_{k'=1}^{N_p} \exp(s_{i,N+k'}(\boldsymbol{X}))} f_{N+j'}(\boldsymbol{X}). \quad (6)$$

Sparse MoE

Sparse Prompt Expert Selection. Each prompt expert $f_{N+j'}$ is now associated with a single unified score $\tilde{s}_{j'}$, enabling the efficient selection of the top-K most relevant experts using Equation (4). This yields the active expert set $K_{\boldsymbol{X}}$, and Equation (8) becomes:

$$\hat{\boldsymbol{h}}_{l,i} = \sum_{j=1}^{N} \frac{\exp(s_{i,j}(\boldsymbol{X}))}{\sum_{k=1}^{N} \exp(s_{i,k}(\boldsymbol{X})) + \sum_{k' \in K_{\boldsymbol{X}}} \exp(\tilde{s}_{k'}(\boldsymbol{X}))} f_j(\boldsymbol{X}) + \sum_{j' \in K_{\boldsymbol{X}}} \frac{\exp(\tilde{s}_{j'}(\boldsymbol{X}))}{\sum_{k=1}^{N} \exp(s_{i,k}(\boldsymbol{X})) + \sum_{k' \in K_{\boldsymbol{X}}} \exp(\tilde{s}_{k'}(\boldsymbol{X}))} f_{N+j'}(\boldsymbol{X}). \quad (9)$$

Reason/Effect

New “prompt” experts that can be trained for task specialisation

To prevent cross task interference while updating weights for each new task

Key mechanisms: SMoPE Model

Adaptive Noise

Adaptive Noise Mechanism. To address this issue, we introduce a novel adaptive noise mechanism that promotes more balanced expert utilization in continual learning, as follows:

$$K_{\boldsymbol{X}} = \underset{S \subseteq \{1,\ldots,N_p\}:|S|=K}{arg\,max} \sum_{j' \in S} (\tilde{s}_{j'}(\boldsymbol{X}) - \epsilon_{j'}), \tag{12}$$

$$\epsilon_{j'} = \begin{cases} \epsilon \cdot (\max_j \tilde{s}_j(\boldsymbol{X}) - \min_j \tilde{s}_j(\boldsymbol{X})) & \text{if } \texttt{train} \text{ and } F_{j'} \geq \frac{1}{N_p} \sum_{j=1}^{N_p} F_j \\ 0 & \text{otherwise} \end{cases} \tag{13}$$

Router loss

Promoting Expert Specialization. During both training and inference, each MSA layer receives an input $\boldsymbol{X}$ and selects a subset of prompt experts. To facilitate this selection, we encourage expert specialization, whereby each expert focuses on distinct regions of the input space. This enables more reliable routing, as specialized experts are more easily identifiable for a given input. To promote such specialization, we introduce the following objective:

$$\mathcal{L}_{\text{router}} = - \sum_{j' \in K_{\boldsymbol{X}}} \frac{\exp\left(\tilde{s}_{j'}(\boldsymbol{X})\right)}{\sum_{k' \in K_{\boldsymbol{X}}} \exp\left(\tilde{s}_{k'}(\boldsymbol{X})\right) + \sum_{k' \notin K_{\boldsymbol{X}}} \exp\left(\tilde{s}_{k'}(\boldsymbol{X})\right)}. \tag{14}$$

This objective encourages higher scores $\tilde{s}_{j'}(\boldsymbol{X})$ for experts in the selected set $K_{\boldsymbol{X}}$ while suppressing the scores of unselected experts. As a result, experts develop more distinct input-space specializations, reducing redundancy and improving routing accuracy.

Reason/Effect

Prevent few experts dominating across all tasks, so we must penalise frequently used experts

Experts focus on distinct regions of input space, rather than clustering towards similar inputs.

Key mechanisms: SMoPE Model

Prototype Loss

Prefix Keys as Prototypes. From Equation (7), optimizing $\tilde{s}_{j'}(\boldsymbol{X})$ corresponds to updating the prefix key $\boldsymbol{p}_{j'}^K$. Since the score function determines expert activation, *the prefix key effectively defines the region of the input space in which each expert specializes*. Thus, optimizing $\mathcal{L}_{\text{router}}$ requires updating both selected and unselected prefix keys.

However, in continual learning, this poses a challenge: past-task data are unavailable, and updating unselected prefix keys may lead to overwriting prior specializations. To address this, we propose treating *prefix keys from earlier tasks as prototypes*, which serve as implicit memory representations of past input distributions. Specifically, we define the prototype set: $\mathcal{D}_{\text{proto}} = \left\{ \boldsymbol{p}_{\text{old},j'}^K \mid 1 \le j' \le N_p,\ F_{j'} \ge \frac{1}{N_p} \sum_{j=1}^{N_p} F_j \right\}$, where $\boldsymbol{p}_{\text{old},j'}^K$ denotes the prefix key after training on the previous task. Only frequently activated experts are retained to avoid noisy or uninformative prototypes. We then define the prototype-based objective:

$$\mathcal{L}_{\text{proto}} = -\sum_{\boldsymbol{p} \in \mathcal{D}_{\text{proto}}} \sum_{j' \in K_{\boldsymbol{p}}} \frac{\exp\left(\boldsymbol{p}^\top \boldsymbol{p}_{j'}^K\right)}{\sum_{k'=1}^{N_p} \exp\left(\boldsymbol{p}^\top \boldsymbol{p}_{k'}^K\right)}, \tag{15}$$

where $K_{\boldsymbol{p}} = \underset{S \subseteq \{1,\ldots,N_p\}:|S|=K}{arg\,max} \sum_{j' \in S} \left(\boldsymbol{p}^\top \boldsymbol{p}_{j'}^K\right)$ denotes the top-K experts activated by prototype $\boldsymbol{p}$.

While $\mathcal{L}_{\text{router}}$ promotes specialization using current-task data, $\mathcal{L}_{\text{proto}}$ preserves specializations learned from previous tasks, thereby mitigating catastrophic forgetting without relying on past inputs.

Reason/Effect

Use old prefix keys as memory of past inputs top preserve expert specialisation.

Paper Results

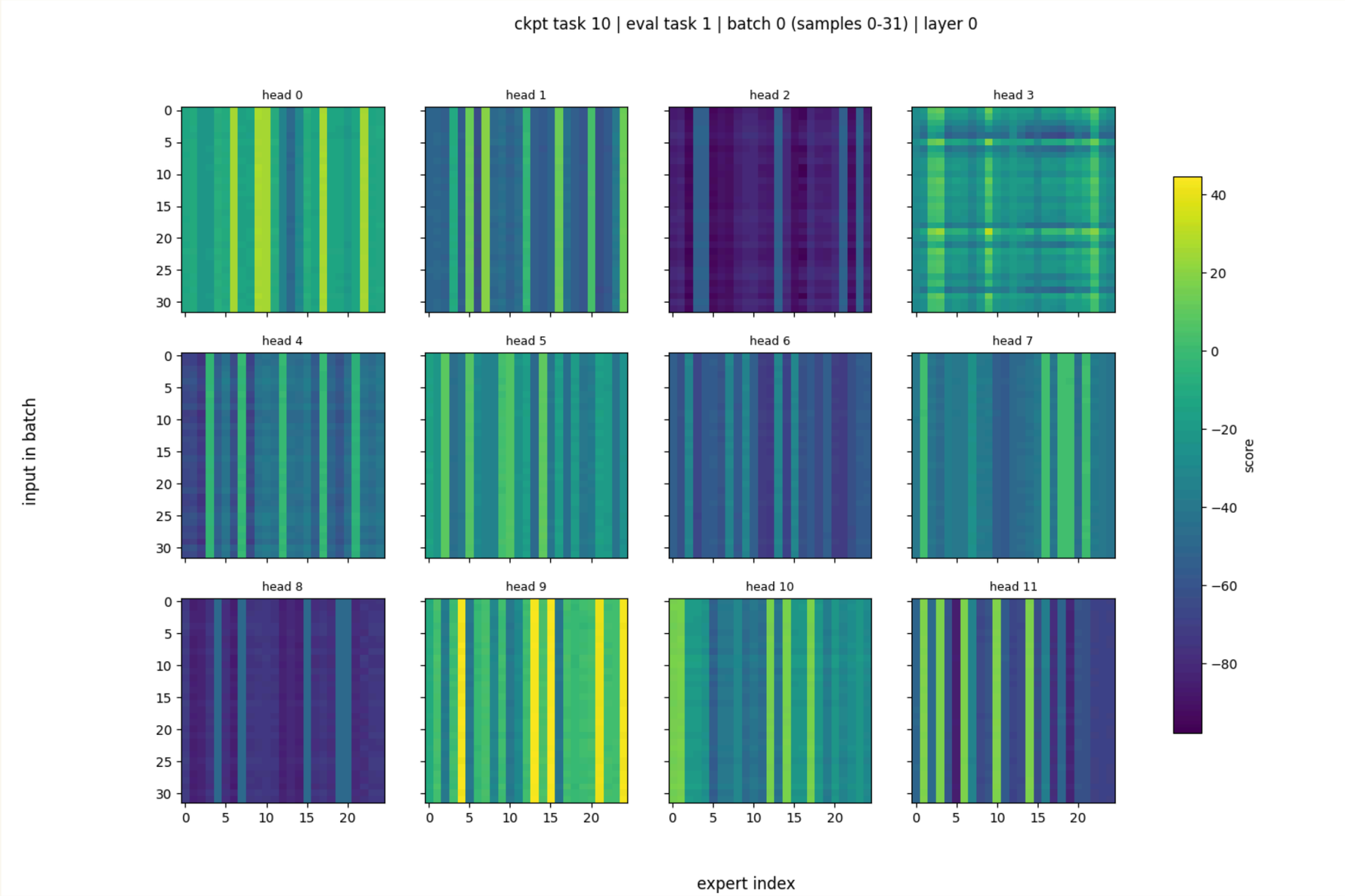
Table 1: Performance comparison on ImageNet-R, CIFAR-100, and CUB-200 (10-task splits). ↑ indicates higher is better. FAA and CAA are averaged over 5 runs. **Bold** denotes the best results, excluding joint training.

Method	ImageNet-R		CIFAR-100		CUB-200	
	FAA (↑)	CAA (↑)	FAA (↑)	CAA (↑)	FAA (↑)	CAA (↑)
Joint-Train	82.06		91.38		88.00	
L2P++	69.29 ± 0.73	78.30 ± 0.69	82.50 ± 1.10	88.96 ± 0.82	77.18 ± 0.71	84.31 ± 0.17
Deep L2P++	71.66 ± 0.64	79.63 ± 0.90	84.30 ± 1.03	90.50 ± 0.69	78.64 ± 0.60	86.14 ± 0.57
DualPrompt	71.32 ± 0.62	78.94 ± 0.72	82.37 ± 0.29	87.10 ± 0.55	77.27 ± 0.31	84.82 ± 0.15
CODA-Prompt	75.45 ± 0.56	81.59 ± 0.82	87.02 ± 0.10	91.61 ± 0.91	76.65 ± 0.70	84.16 ± 0.23
OVOR	75.25 ± 0.21	79.78 ± 0.65	86.91 ± 0.35	91.02 ± 1.00	77.45 ± 0.69	85.81 ± 1.56
HiDe-Prompt	74.25 ± 0.19	79.64 ± 0.53	88.27 ± 0.59	91.38 ± 0.78	85.60 ± 0.05	90.16 ± 0.64
NoRGa	74.39 ± 0.08	79.68 ± 0.41	88.79 ± 0.13	92.06 ± 0.51	85.78 ± 0.24	90.63 ± 0.58
ConvPrompt	77.30 ± 0.45	81.46 ± 0.67	88.76 ± 0.36	92.49 ± 0.89	82.56 ± 0.61	86.68 ± 0.95
CPrompt	77.14 ± 0.11	82.92 ± 0.70	87.82 ± 0.21	92.53 ± 0.23	80.35 ± 0.44	85.66 ± 0.59
VQ-Prompt	78.71 ± 0.22	83.24 ± 0.68	88.73 ± 0.27	92.84 ± 0.73	86.72 ± 0.94	90.33 ± 1.03
SMoPE	**79.32** ± 0.42	**84.39** ± 0.77	**89.23** ± 0.12	**93.67** ± 0.82	**87.43** ± 0.39	**91.11** ± 0.55

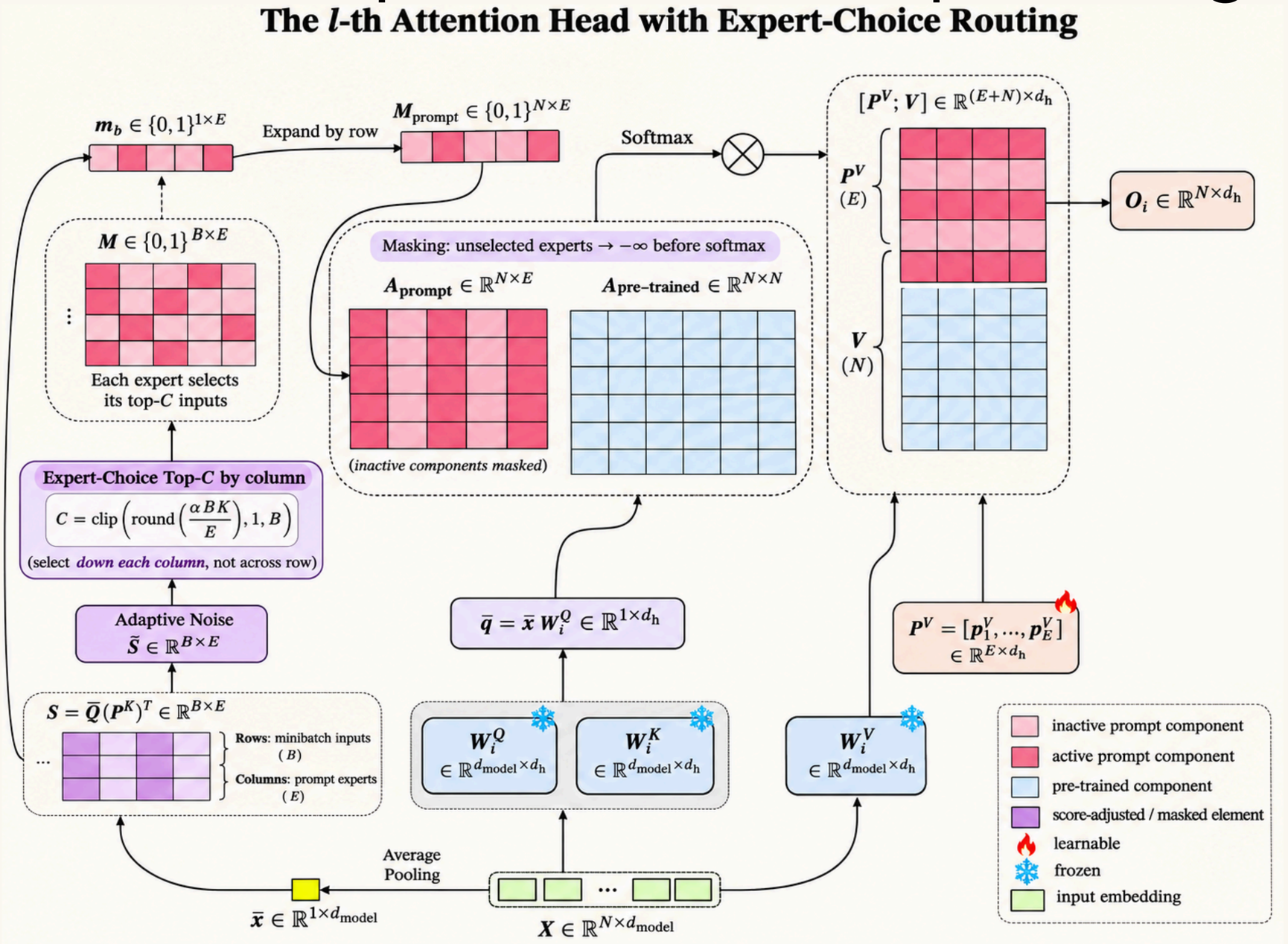
Table 2: Performance comparison on ImageNet-R (10-task split) for self-supervised pre-training, iBOT-1K, and DINO-1K. ↑ indicates higher is better. FAA and CAA are averaged over 5 runs. **Bold** highlights the best results.

```
===Summary of experiment repeats: 2 / 2 ===
acc  | mean: 89.71000000000001 std: 0.19999999999999576
time  | mean: 0.2861376628661767 std: 0.0005992822158030764
fr  | mean: 3.23888888888889 std: 0.283333333333333
FAA | mean: 89.71000000000001 std: 0.19999999999999576
CAA | mean: 92.41366071428573 std: 0.06041468253967963
```

Scores of input vs Experts in SMoPE



Task 1: Expert-Choice Prompt Routing



Task 1: Expert-Choice Prompt Routing

Replace the input-wise Top-K selection with an expert-choice mechanism in which each prompt expert selects a fixed number of the highest-scoring inputs from the minibatch.

```python
def expert_choice_mask(scores, topk):
    """scores (B, heads, 1, experts) -> bool mask of the same shape.
    For every head and expert: mark the `capacity` inputs of the minibatch with the highest score."""
    B, H, _, E = scores.shape
    best = scores.squeeze(2).topk(expert_capacity(B, E, topk), dim=0).indices  # (capacity, H, E)
    mask = torch.zeros(B, H, E, dtype=torch.bool, device=scores.device)
    mask.scatter_(0, best, True)
    return mask.unsqueeze(2)
```

```python
    if topk > 0 and prompt is not None and reduce_query and ROUTING == "expert_choice":
        # EXPERT CHOICE: same noise-adjusted score as SMoPE, but ranked over the inputs of the batch.
        eps_decay = eps_decay.unsqueeze(1)
        with torch.no_grad():
            score_range = prompt_score_.max(dim=-1, keepdim=True).values - prompt_score_.min(
                dim=-1, keepdim=True
            ).values
        prompt_score_label_ = prompt_score_ - eps_decay * score_range  # (B, num_heads, 1, num_prompt)
        chosen = expert_choice_mask(prompt_score_label_.detach(), topk)  # bool, same shape
        off = (~chosen).expand(-1, -1, N, -1)  # experts that did NOT choose this input
        attn = torch.cat(
            [attn[..., :prompt_length].masked_fill(off, float("-inf")), attn[..., prompt_length:]],
            dim=-1,
        )  # switched-off experts get zero attention weight
```

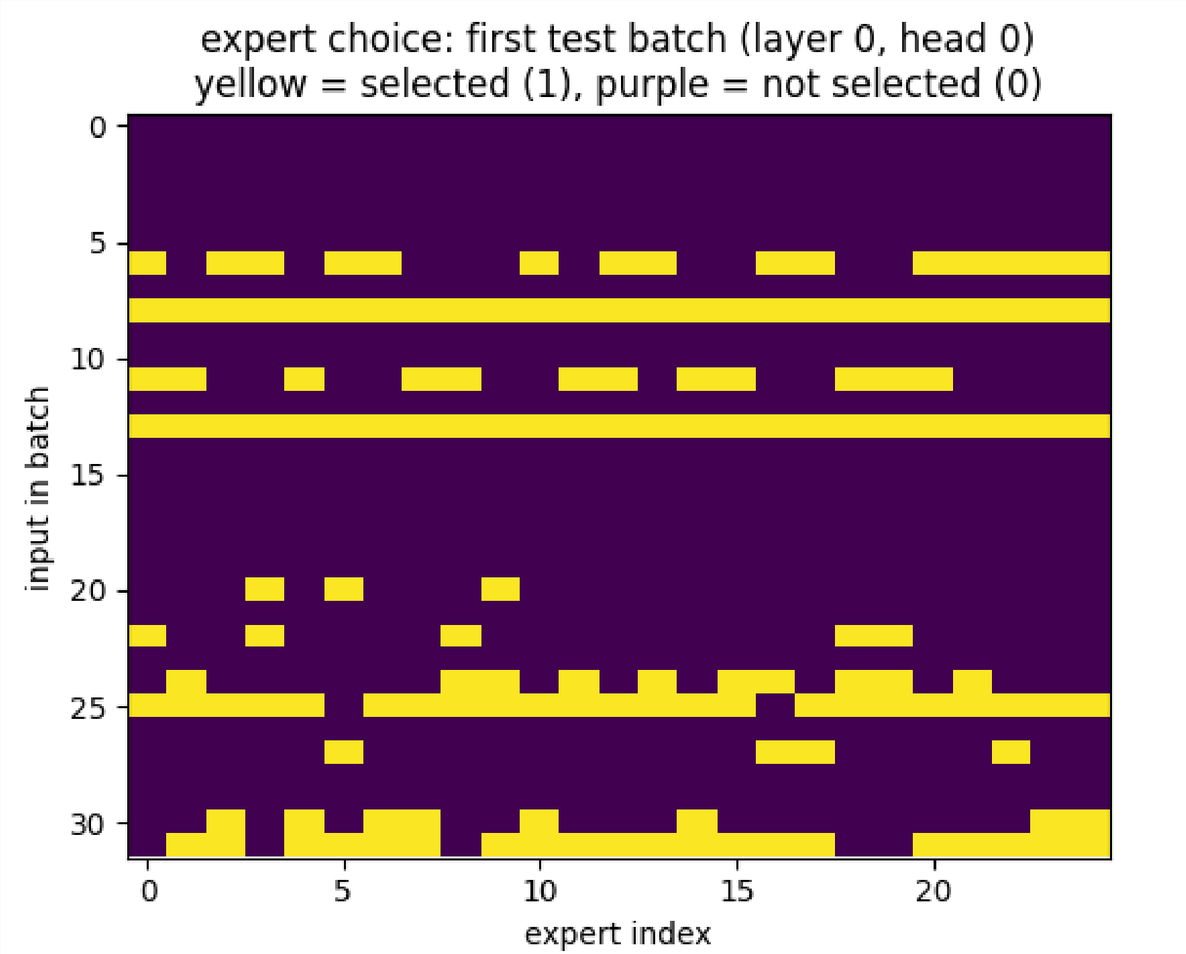
results:

	SMoPE top-K	expert choice
FAA (%)	89.71 +- 0.20	87.42 +- 0.05
CAA (%)	92.41 +- 0.06	90.44 +- 0.13
active experts per input	5.00	4.70
inputs with 0 experts (%)	0.00	66.53
balance CV (0 = even)	1.998	0.000
prompt MACs per image (M)	4.68	4.40

```
(smope) dse_413@workstattion15:~/Documents/ADL/SMoPE-master$
fr |  mean:  2.6929107142857127 std :  0.9824288820344118
```

macs = L * H * head_dim * (E + active * (1 + n_tokens))

Task 1: Expert-Choice Prompt Routing Visualisation



Task 2: Orthogonal Prompt Expert Specialization

Subtask

SMoPE does not explicitly decorrelate expert prefix keys.

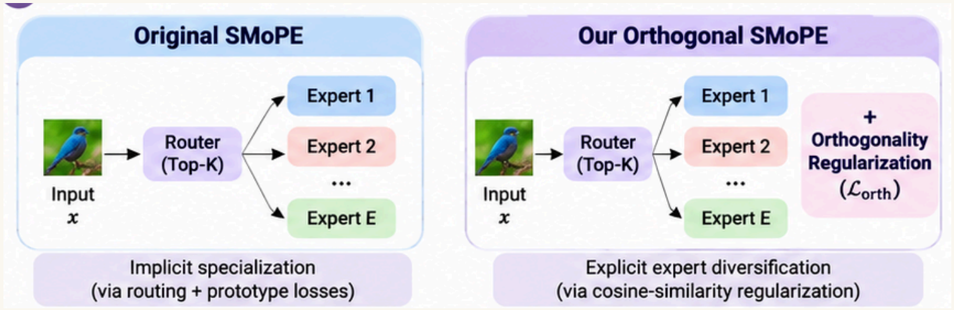
task: introduction of Orthogonal Prompt Expert Regularization, which minimizes pairwise cosine similarity between expert prefix keys to encourage distinct representation directions, reduce redundancy, and promote more diverse routing regions

Our solution

```
    if self.mu_orth > 0:
        loss += self.mu_orth * self.orth_loss()

    return loss

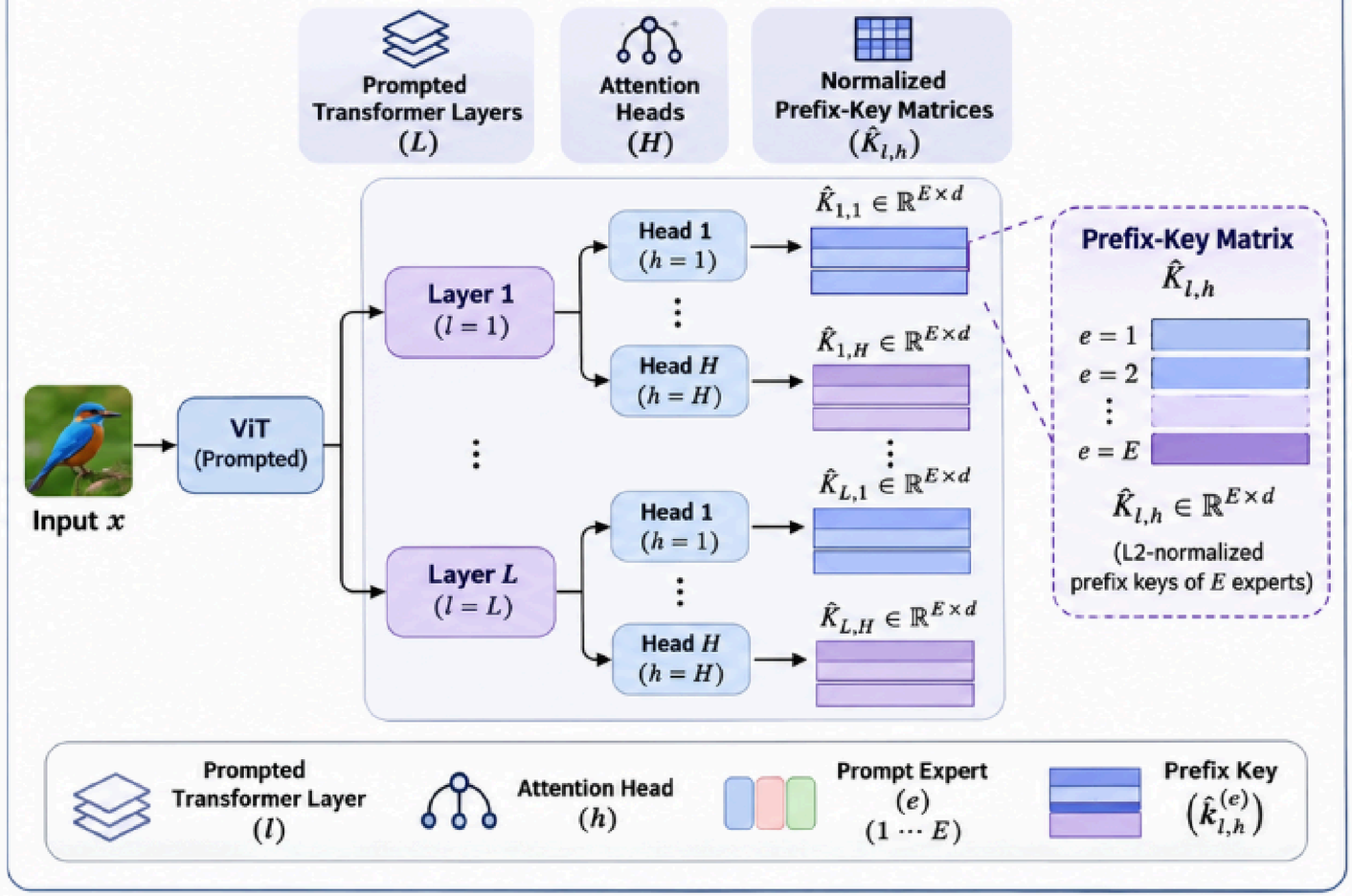
def orth_loss(self):
    """Orthogonal prompt expert regularization: for every layer and head, the prefix keys of the
    different experts should have cosine similarity 0 (penalty = mean squared off-diagonal cosine)."""
    loss = 0.0
    for e in self.e_layers:
        for h in range(self.num_heads):
            pk = torch.cat([getattr(self, f"e_pk_{e}_{l}_{h}") for l in range(self.num_experts)], dim=0)
            pk = F.normalize(pk, dim=-1)  # (num_experts, head_dim), unit length
            cos = pk @ pk.t()  # (num_experts, num_experts) pairwise cosine similarity
            off_diag = cos - torch.eye(self.num_experts, device=cos.device)  # remove the diagonal (= 1)
            loss = loss + (off_diag**2).sum() / (self.num_experts * (self.num_experts - 1))
    return loss / (len(self.e_layers) * self.num_heads)
```



Run	FAA (%)	CAA (%)	Mean |cos|	Max cos	Act. CV
Original	89.71 ± 0.20	92.41 ± 0.06	0.594	1.000	1.998
0.001	89.90 ± 0.19	92.54 ± 0.05	0.139	1.000	1.994
0.1	89.65 ± 0.02	92.40 ± 0.18	0.072	0.997	1.988
1	89.74 ± 0.38	92.44 ± 0.10	0.053	0.875	1.982

```
fr |  mean:  2.6993769841269835 std :  1.1467229827465337
```

From Prompted Layers to Normalized Prefix-Key Matrices



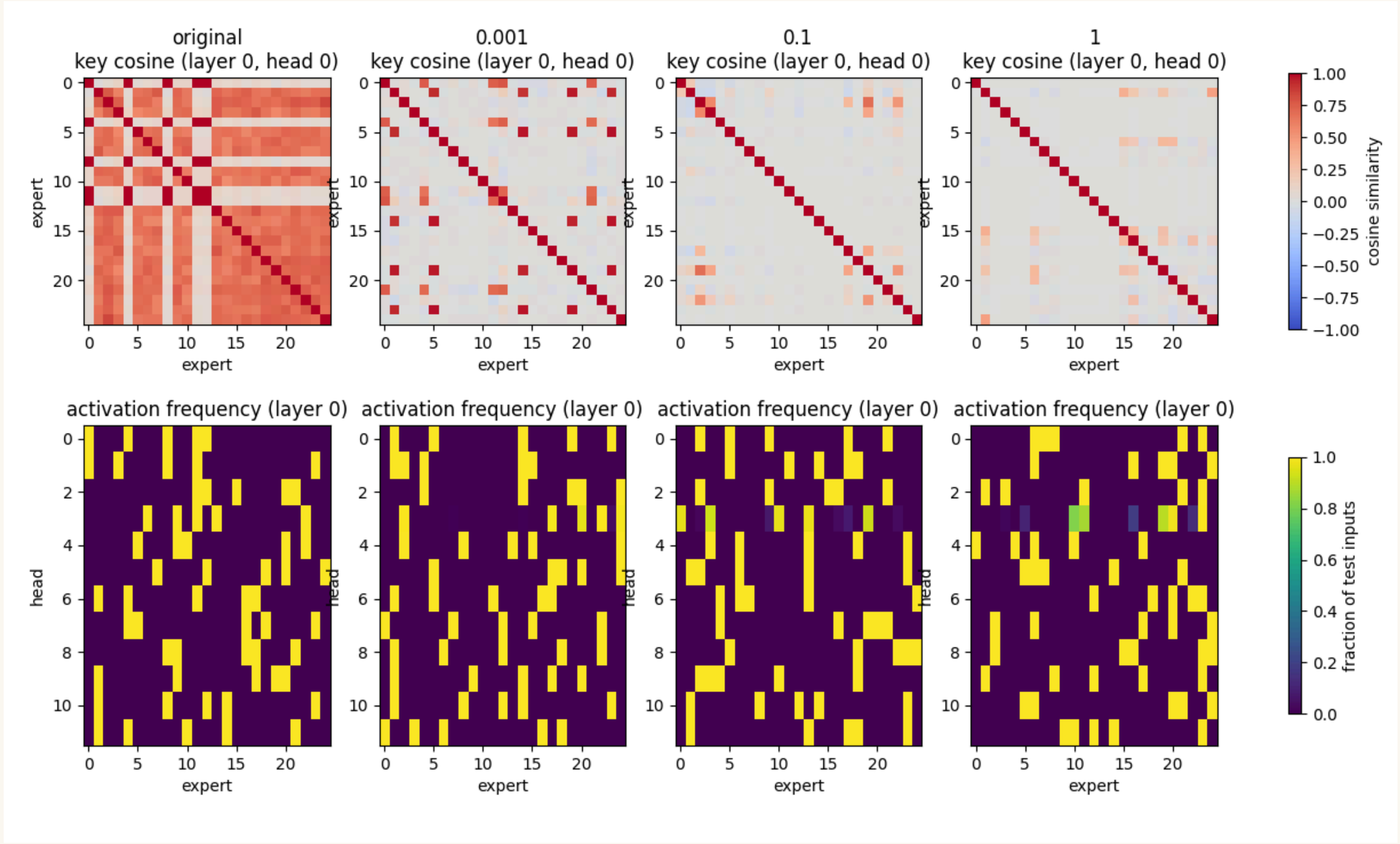
Notation

L	L: Number of prompted Transformer layers
H	H: Number of attention heads per layer
E	E: Total number of prompt experts
$\hat{K}_{l,h}$	$\hat{K}_{l,h}$: Matrix of L2-normalized expert prefix keys
I_E	I_E: Identity matrix of size $E \times E$
$\|\cdot\|_F$	$\|\cdot\|_F$: Frobenius norm
λ_{orth}	λ_{orth}: Orthogonality regularization coefficient
$\mathcal{L}_{\text{SMoPE}}$	$\mathcal{L}_{\text{SMoPE}}$: Original SMoPE training objective

Orthogonality Regularization

$$\mathcal{L}_{\text{orth}} = \frac{1}{LH} \sum_{l} \sum_{h} \frac{\left\| \hat{K}_{l,h} \hat{K}_{l,h}^{T} - I_E \right\|_F^2}{E(E-1)}$$

Effect of Orthogonality



Conclusion

Model/Method	FAA	CAA	Forgetting
SMoPE	89.71	92.41	3.23
Expert Choice	87.42	90.44	2.69
Orthogonality(0.0(	**89.9**	**92.54**	**2.69**

PROJECT 9

Thank You

GROUP 18

Poorvai Chandrasen

Palak Kumari

Shruti Wadatkar

CHANGES AND ADD-ONS IN “vit.py”

(TASK 1)

Added routing configuration

```
22      ROUTING = os.environ.get("SMOPE_ROUTING", "topk")
23      CAPACITY_FACTOR = float(os.environ.get("SMOPE_CAPACITY", "1.0"))
```

Added the expert-capacity function

```
def expert_capacity(batch_size, num_experts, topk):
    """Inputs taken by each expert = B*K/E, so about B*K (input, expert) pairs in total, like top-K."""
    c = int(batch_size * topk * CAPACITY_FACTOR / num_experts + 0.5)
    return min(batch_size, max(1, c))
```

Added selection across the batch dimension

```
def expert_choice_mask(scores, topk):
    """scores (B, heads, 1, experts) -> bool mask of the same shape.
    For every head and expert: mark the `capacity` inputs of the minibatch with the highest score."""
    B, H, _, E = scores.shape
    best = scores.squeeze(2).topk(expert_capacity(B, E, topk), dim=0).indices  # (capacity, H, E)
    mask = torch.zeros(B, H, E, dtype=torch.bool, device=scores.device)
    mask.scatter_(0, best, True)
    return mask.unsqueeze(2)
```

Added an expert-choice branch inside attention

```
174             if topk > 0 and prompt is not None and reduce_query and ROUTING == "expert_choice":
175                 # EXPERT CHOICE: same noise-adjusted score as SMoPE, but ranked over the inputs of the batch.
176                 eps_decay = eps_decay.unsqueeze(1)
177                 with torch.no_grad():
178                     score_range = prompt_score_.max(dim=-1, keepdim=True).values - prompt_score_.min(
179                         dim=-1, keepdim=True
180                     ).values
181                 prompt_score_label_ = prompt_score_ - eps_decay * score_range  # (B, num_heads, 1, num_prompt)
182                 chosen = expert_choice_mask(prompt_score_label_.detach(), topk)  # bool, same shape
183                 off = (~chosen).expand(-1, -1, N, -1)  # experts that did NOT choose this input
184                 attn = torch.cat(
185                     [attn[..., :prompt_length].masked_fill(off, float("-inf")), attn[..., prompt_length:]],
186                     dim=-1,
187                 )  # switched-off experts get zero attention weight
```

Instead of just:

```
150             if topk > 0 and prompt is not None and reduce_query:
151                 # Get topK among all MoE models
152                 _, num_heads, N, _ = attn.size()
153                 eps_decay = eps_decay.unsqueeze(1)
```

CHANGES AND ADD-ONS IN “zoo.py”

Added the orthogonality coefficient

```
26              # strength of the orthogonality regularization on the prefix keys (6th value, 0 = original SMoPE)
27              self.mu_orth = float(prompt_param[5]) if len(prompt_param) > 5 else 0.0
```

Added the penalty to the existing auxiliary loss

```
147                 if self.mu_orth > 0:
148                     loss += self.mu_orth * self.orth_loss()
```

Added OnePrompt.orth_loss()

```python
    def orth_loss(self):
        """Orthogonal prompt expert regularization: for every layer and head, the prefix keys of the
        different experts should have cosine similarity 0 (penalty = mean squared off-diagonal cosine)."""
        loss = 0.0
        for e in self.e_layers:
            for h in range(self.num_heads):
                pk = torch.cat([getattr(self, f"e_pk_{e}_{l}_{h}") for l in range(self.num_experts)], dim=0)
                pk = F.normalize(pk, dim=-1)  # (num_experts, head_dim), unit length
                cos = pk @ pk.t()  # (num_experts, num_experts) pairwise cosine similarity
                off_diag = cos - torch.eye(self.num_experts, device=cos.device)  # remove the diagonal (= 1)
                loss = loss + (off_diag**2).sum() / (self.num_experts * (self.num_experts - 1))
        return loss / (len(self.e_layers) * self.num_heads)
```

mathematical function :

$$L_{\text{orth}} = \frac{1}{LH} \sum_{\ell,h} \frac{\sum_{i \neq j} \cos(p_i^K, p_j^K)^2}{E(E-1)}$$

Task 1: Expert-Choice Prompt Routing

Task 1 — Expert-Choice (Ours)

Score: $S_{b,e}^{(\ell,h)} = \langle \bar{q}_b^{(\ell,h)}, p_e^{K,(\ell,h)} \rangle, \quad S \in \mathbb{R}^{B\times H\times 1\times E}$

Capacity: $C = \text{clip}\left(\text{round}\left(\frac{\alpha BK}{E}\right), 1, B\right)$

Selection: $A_{b,e}^{EC} = 1[b \in \text{TopC}(\tilde{S}_{:,e}, C)]$

Previous SMoPE — Input-wise Top-K

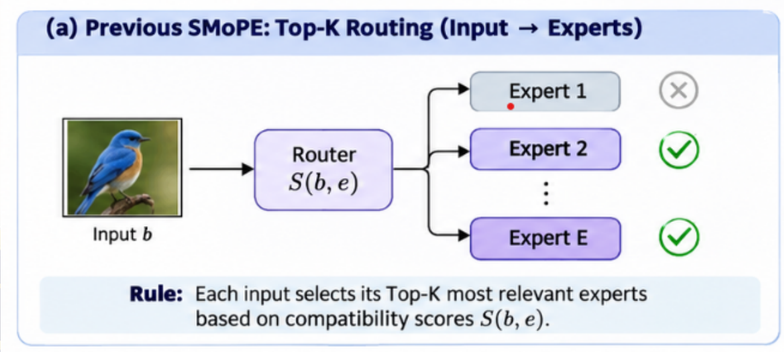
Score: $S_{b,e}^{(\ell,h)} = \langle \bar{q}_b^{(\ell,h)}, p_e^{K,(\ell,h)} \rangle, \quad S \in \mathbb{R}^{B\times H\times 1\times E}$

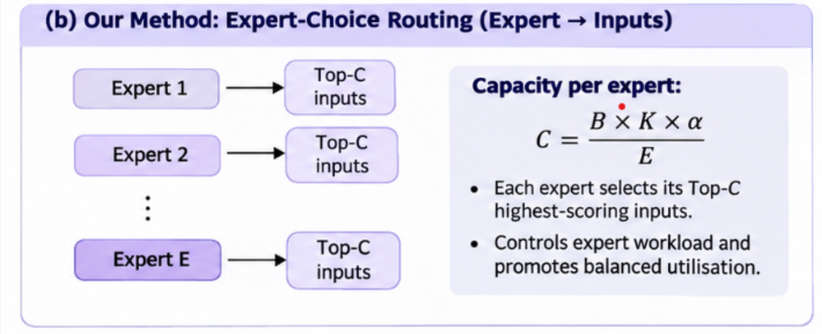
Selection: $A_{b,e}^{TopK} = 1[e \in \text{TopK}(\tilde{S}_{b,:}, K)]$

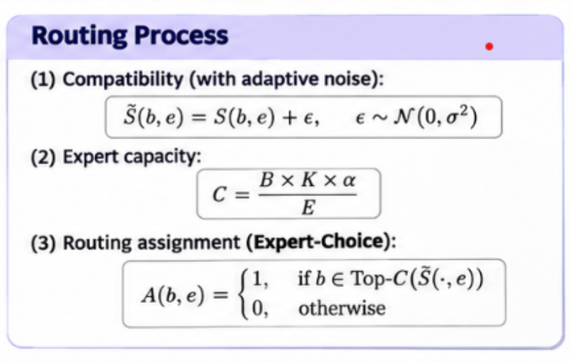
Previous SMoPE: Input → Top-K Experts
Our Expert-Choice: Expert → Top-C Inputs

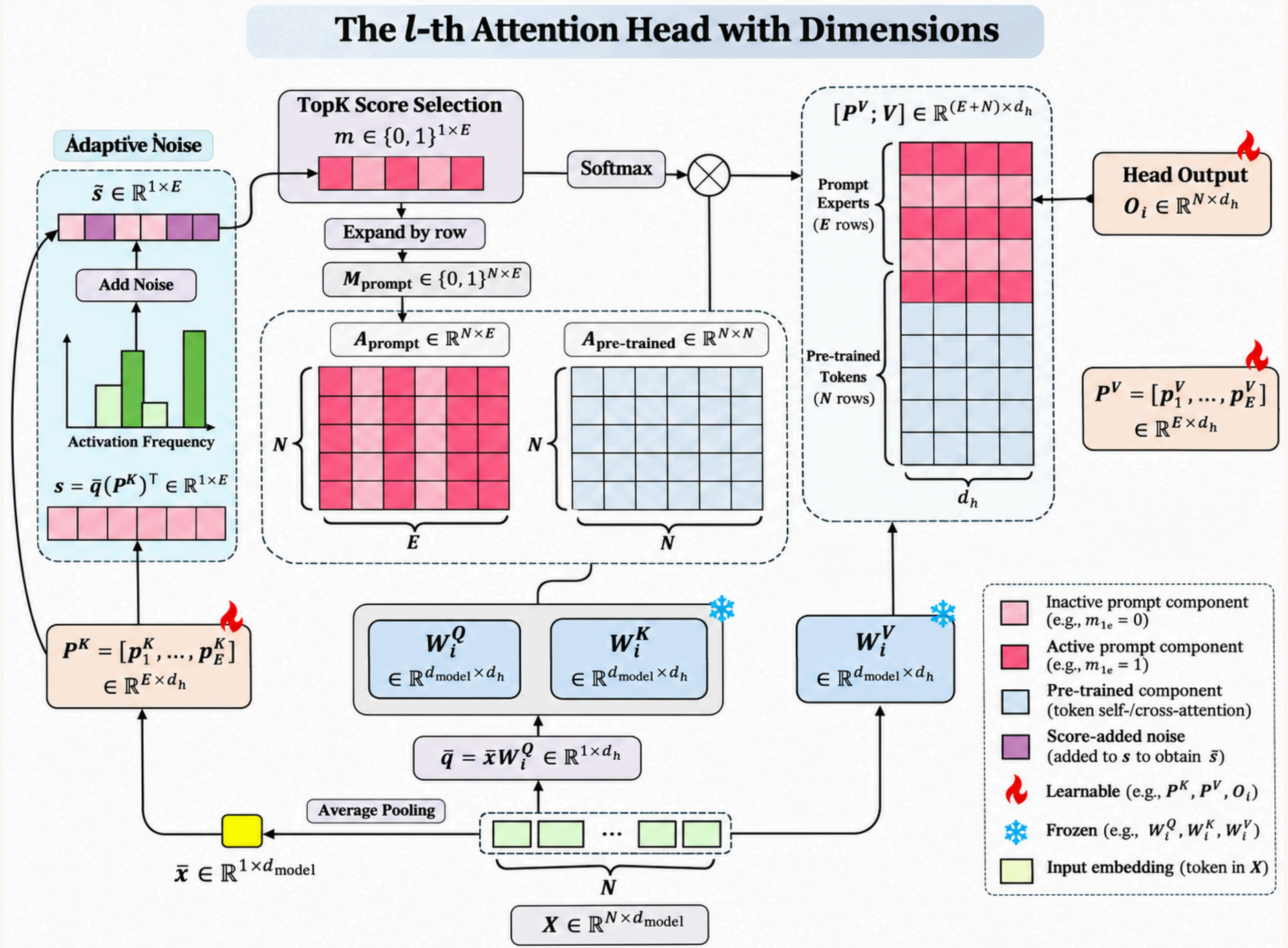
- C = (Batch Size × K × Capacity Factor) / Total Experts
- where
- B: Total number of inputs (images) in a minibatch.
- b: Index of an individual input in the minibatch.
- E: Total number of prompt experts.
- e: Index of an individual prompt expert.
- α: Capacity scaling factor controlling expert workload.
- S: Raw input–expert compatibility score.
- S̃: Adaptive-noise–adjusted compatibility score.
- A: Binary routing assignment (1 = selected, 0 = not selected).

The main aim of this experiment is to improve expert utilisation, but the limitation is that it looks at both past and future tokens to decide which expert to use.








The l-th Attention Head with Dimensions
Adaptive Noise
$\tilde{s} \in \mathbb{R}^{1\times E}$
Add Noise
Activation Frequency
$s = \bar{q}(P^K)^\top \in \mathbb{R}^{1\times E}$
TopK Score Selection
$m \in \{0,1\}^{1\times E}$
Softmax
Expand by row
$M_{\text{prompt}} \in \{0,1\}^{N\times E}$
$A_{\text{prompt}} \in \mathbb{R}^{N\times E}$
$A_{\text{pre-trained}} \in \mathbb{R}^{N\times N}$
N
E
N
$[P^V; V] \in \mathbb{R}^{(E+N)\times d_h}$
Prompt Experts (E rows)
Pre-trained Tokens (N rows)
d_h
Head Output
$O_i \in \mathbb{R}^{N\times d_h}$
$P^V = [p_1^V, \ldots, p_E^V]$
$\in \mathbb{R}^{E\times d_h}$
$P^K = [p_1^K, \ldots, p_E^K]$
$\in \mathbb{R}^{E\times d_h}$
W_i^Q
$\in \mathbb{R}^{d_{\text{model}}\times d_h}$
W_i^K
$\in \mathbb{R}^{d_{\text{model}}\times d_h}$
W_i^V
$\in \mathbb{R}^{d_{\text{model}}\times d_h}$
$\bar{q} = \bar{x}W_i^Q \in \mathbb{R}^{1\times d_h}$
Average Pooling
$\bar{x} \in \mathbb{R}^{1\times d_{\text{model}}}$
N
$X \in \mathbb{R}^{N\times d_{\text{model}}}$
Inactive prompt component (e.g., $m_{1e} = 0$)
Active prompt component (e.g., $m_{1e} = 1$)
Pre-trained component (token self-/cross-attention)
Score-added noise (added to s to obtain $\tilde{s}$)
Learnable (e.g., P^K, P^V, O_i)
Frozen (e.g., W_i^Q, W_i^K, W_i^V)
Input embedding (token in X)